AI Bootcamp Week 3: Model Invocation, Adversarial Agents, and Bounded Agents

Monday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 3, the candidate can:

- Explain the DLP automation spectrum and what evidence is required to move up each step
- Explain why the DLP defaults to skills with `disable-model-invocation: true`, and articulate the three graduation criteria for enabling model invocation
- Write invocation test cases (positive trigger, negative trigger, quality) and use them to make a documented enable-or-gap decision on a real skill
- Build an adversarial agent that challenges a skill's output, and explain how it complements (not duplicates) a Python evaluation script
- Run an evaluator-vs-adversarial comparison and use the delta to improve either the EC or the skill
- Compose skills into a bounded agent: a multi-step pipeline run in one IDE session with evaluation and adversarial review at key steps
- Explain context window degradation, why bounded agents hit a ceiling, and what a custom harness is (awareness level; building one is Stage 4 work)
- Describe the IIOS stage framework: where DLP Completion sits (Stage 3, Task) and what Stage 4 (Workflow) requires

© 2026 Improving

TIMING

- **Monday:** 1:00 PM – 5:00 PM (travel morning: 4 hours)
- **Tuesday–Thursday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each)
- **Friday:** 9:00 AM – 12:00 PM (travel afternoon: 3 hours)

© 2026 Improving

MONDAY (1:00 PM – 5:00 PM): THE SPECTRUM AND THE INVOCATION DECISION

1:00 – 2:30 PM: INVOCATION DISABLED VS. ENABLED

Every skill you built in Weeks 1 and 2 has `disable-model-invocation: true` in its frontmatter. You run it via slash command, at a time you choose. This is the default mode during a DLP engagement, and it is a deliberate design decision, not a limitation.

Why the DLP defaults to invocation-disabled:

A skill with invocation disabled only runs when you run it. If the output is wrong, only you are affected. You learn, you iterate, you improve it. That is the EDD discipline.

A skill with invocation enabled runs when the model decides to run it. If the model invokes the skill at the wrong time, it may produce output that downstream processes act on before anyone reviews it. And because different people on a team may have different expectations about when the skill should trigger, premature auto-invocation creates invisible disagreement about workflow that surfaces as confusing or inconsistent outputs.

Reconnect to Prompt/Harness/Model: the harness controls invocation, not the model. `disable-model-invocation: true` tells the harness "only run this skill when a human explicitly invokes it." Flipping the flag tells the harness "the model may decide to run this skill based on context." The model still cannot do anything the harness does not allow; you are changing what the harness allows.

The graduation frame: enabling model invocation is not a conversion or a migration. It is a configuration decision you earn with evidence. The old question was "should this be a skill or a command?" Everything is already a skill. The only question is "has this skill earned auto-invocation?"

Exercise: Review two of your Week 1 or Week 2 skills. For each, answer in writing:

- Could this skill be invoked at the wrong time and cause harm (wrong output, downstream dependency failures)?
- Does the right invocation time depend on things the model might judge differently from a human?
- If two team members had different opinions about when this skill should run, what would break?

2:30 – 4:00 PM: THE THREE GRADUATION CRITERIA

A skill is ready for model invocation when the team has agreed, in writing, on:

1. **Exact trigger inputs.** What specific input conditions should cause the model to invoke this skill? Not "when it seems relevant." Specific, describable input conditions. Your evaluation history tells you exactly which inputs produce good output; those are your candidate trigger conditions.
2. **Output specification.** What does the skill do when triggered? What output does it produce? This must be specific enough that you can verify the skill triggered correctly.

© 2026 Improving

3. **At least 3 verified trigger test cases.** At least 3 specific scenarios where the skill SHOULD invoke, verified to confirm that (a) the model actually does invoke it when those conditions are present, and (b) the output meets the specification from criterion 2. If you cannot name three specific scenarios and verify both invocation and output quality, you do not understand the invocation conditions well enough to automate it.

The most important criterion is #3. The difficulty of naming three verifiable trigger scenarios is the signal. If it is hard, you need more evaluation, not less.

The EDD prerequisite: a skill should have at least 3 documented iteration cycles before model invocation is enabled. If it does not have that history, the prerequisite is more EDD work, not graduation.

Exercise: Take the two skills from the previous exercise. For each, attempt to write all three graduation criteria in full. If you cannot write a criterion, write down specifically what is missing. If you cannot write it, you do not have consensus; you have an opinion.

4:00 – 5:00 PM: OVER-INVOCATION AND UNDER-INVOCATION

When skills go wrong with invocation, there are two failure modes:

- **Under-invocation:** the skill does not run when it should. A human does the work manually. Annoying, recoverable.
- **Over-invocation:** the skill runs when it should not. Output gets produced, and may get committed or acted on, before anyone notices. This is the most common and most urgent invocation failure in the field.

Key insight: Over-invocation is more dangerous than under-invocation. An un-invoked skill means a human does the work. An over-invoked skill means a machine did work no one expected, and the evidence may already be in the codebase.

Defense: negative invocation test cases. Scenarios where the skill should NOT invoke, verified to confirm it stays silent. Every over-invocation incident in the field becomes a new negative test case and a tightened invocation boundary.

© 2026 Improving